# A divergent program with every possible output in a model of Peano arithmetic

A proof and Lean formalization plan
Prepared for Vladimir Reshetnikov

November 9, 2026

**Abstract**

There is one partial computable program which diverges on every standard input, while for each standard natural number $n$ some first-order model of Peano arithmetic believes that the program halts on input 0 with output $n$. We prove this directly by self-referential proof search, using only consistency, verification of finite computations, and completeness. We also explain the immediate connection with the universal algorithm in the reference supplied by the user. A concrete Lean statement is given using Mathlib program codes and Foundation's actual first-order satisfiability predicate. Accompanying source files isolate the short proof and apply Mathlib's existing recursion theorem. The remaining arithmetization and proof-search obligations are specified explicitly. The mathematical proof is complete; the concrete PA formalization is a detailed implementation outline, and the supplied Lean files have not been compiler-checked in the authoring environment.

## Contents

**1 The precise statement** **2**

**2 A direct proof by self-referential proof search** **2**
2.1 The three background ingredients . . . 2
2.2 The algorithm . . . 3
2.3 What the proof does and does not require . . . 4

**3 Connection with the supplied universal-algorithm reference** **4**

**4 How the nonstandard model supports the computation** **5**

**5 A concrete Lean formalization of the statement** **5**
5.1 Use first-order syntax and genuine model existence . . . 5
5.2 One fixed universal computation formula . . . 5
5.3 Make the representation contract explicit . . . 6

**6 Existing results located in ProveIt and its dependencies** **7**
6.1 Computability and self-reference . . . 7
6.2 Arithmetic representation and numeral substitution . . . 7
6.3 Consistency of an extension and existence of a model . . . 7
6.4 Two implementation hazards . . . 8

**7 The supplied Lean proof files** **8**
7.1 The logic-only core . . . 8
7.2 The assembly really uses the recursion theorem . . . 8

**8 A clear route to a complete Lean proof** **9**
8.1 Stage 1: validate the chosen computation presentation . . . . . 9
8.2 Stage 2: prove functionality under PA . . . . . 9
8.3 Stage 3: implement uniform proof search . . . . . 10
8.4 Stage 4: instantiate logic and model existence . . . . . 10
8.5 Stage 5: compile and audit . . . . . 10

**9 Optional strengthening: an internally total constant function** **11**

**10 Delivery and verification status** **11**

# 1 The precise statement

Following the user's clarification, computability here means *partial* computability. The standard computation at 0 does not terminate. Thus the external function has type $\mathbb{N} \rightharpoonup \mathbb{N}$, represented in Lean by `Nat -> Part Nat`, usually written $\mathbb{N}$ `→.` $\mathbb{N}$. The expression "$f(0) = n$ holds in $M$" means that $M$ satisfies a fixed arithmetical sentence describing the computation of a particular program.

Fix an effective coding of deterministic programs and a uniform arithmetic formula $\mathrm{Halt}(e, x, y)$ describing that program $e$ halts on input $x$ with output $y$. In the usual trace presentation it is an existential assertion that a coded finite computation has the prescribed initial configuration and output. For standard $e, x, y$ write

$$H_e(x, y) := \mathrm{Halt}(\overline{e}, \overline{x}, \overline{y}).$$

The numerals are part of the object-language sentence. The symbol $n$ in the quantifier below ranges over *external standard* natural numbers.

**Theorem 1.1** (Arbitrary possible output)**.** *There is a standard program code $e$ for a partial computable function $\varphi_e : \mathbb{N} \rightharpoonup \mathbb{N}$ such that*

$$\forall x \in \mathbb{N} \quad \varphi_e(x) \uparrow, \tag{1}$$
$$\forall n \in \mathbb{N} \quad \exists M \ (M \models \mathsf{PA} \text{ and } M \models H_e(0, n)). \tag{2}$$

*Here $\uparrow$ denotes divergence. The same program code and the same arithmetization are used for all $n$.*

More generally, the construction works for every consistent computably axiomatized extension $T$ of PA, with the search performed over proofs of $T$. For the requested case, the standard natural-number structure is a model of PA in the metatheory and gives the required consistency. Nothing in the argument asserts that PA proves its own consistency.

The formal witness must retain the program code. Many different programs compute the empty partial function in the standard model, and their arithmetical computation assertions can behave differently in nonstandard models. Equality of the external partial functions alone does not identify the sentences to interpret inside a model.

# 2 A direct proof by self-referential proof search

## 2.1 The three background ingredients

The following standard facts suffice.

**Effective syntax and proof checking.** PA has an effective proof system. Numeral construction, formula substitution, negation, and checking whether a finite proof has a given conclusion are computable. One can therefore search uniformly over pairs $(p, n)$ for a PA proof $p$ of $\neg H_e(0, n)$, treating $e$ as an ordinary input parameter.

**Verification of actual finite computations.** If the standard computation of program $e$ on input $x$ actually halts with output $y$, then

$$\mathsf{PA} \vdash H_e(x, y). \tag{3}$$

A genuine finite computation can be checked step by step. Equivalently, its computation sentence is a true $\Sigma_1^0$ sentence, and PA proves all such sentences. This is a statement about standard finite computations; it is not an internal reflection principle for all PA proofs.

**Kleene's recursion theorem.** For a partial computable operation $Q(c, x)$, with $c$ a program code, there is a code $e$ such that

$$\varphi_e(x) \simeq Q(e, x) \qquad \text{for every standard } x. \tag{4}$$

Equality $\simeq$ includes agreement about divergence. The theorem is already available in Mathlib as `Nat.Partrec.Code.fixed_pointâĆĆ` [4].

## 2.2 The algorithm

For a supplied code $c$, let $Q(c, x)$ ignore $x$ and run the following ordinary proof search. Use any fixed computable enumeration of pairs of natural numbers, such as unpairing successive natural numbers.

```
Q(c, x):
    for q = 0, 1, 2, ...:
        (p, n) := unpair(q)
        if p is a PA proof of NOT Halt(c, 0, n):
            return n
```

The procedure is partial computable uniformly in $c$ and $x$. Apply the recursion theorem to obtain $e$ satisfying

$$\varphi_e = Q(e, \cdot).$$

This makes the program search for proofs excluding its own outputs. The recursion theorem removes the apparent circularity: the parameterized algorithm $Q$ is defined before any fixed point is chosen.

**Lemma 2.1.** *Assuming* $\mathrm{Con}(\mathsf{PA})$*, the standard computation* $\varphi_e(0)$ *does not halt.*

*Proof.* Suppose that it halts with output $k$. By the fixed-point equation, $Q(e, 0)$ halts with output $k$. The definition of the search then supplies a genuine finite PA proof of $\neg H_e(0, k)$. On the other hand, the actual finite computation of $e$ supplies, by (3), a PA proof of $H_e(0, k)$. These two proofs contradict consistency. □

**Lemma 2.2.** *For every standard* $n$, $\mathsf{PA} \nvdash \neg H_e(0, n)$.

*Proof.* If such a proof existed, exhaustive search would eventually find a proof of the prescribed form. It might first find a proof referring to another output $k$, but it would certainly terminate. The fixed-point equation would then make $\varphi_e(0)$ terminate, contradicting Lemma 2.1. □

*Remark* 2.3 (The first-proof detail). A proof excluding $n$ does *not* necessarily make the program output $n$. An earlier accepted proof can concern another value $k$. The contradiction must use the value actually selected by the search and the proof selected with it. Both the mathematical argument and the supplied Lean assembly preserve this distinction.

*Proof of Theorem 1.1.* The search ignores its input. Its fixed point is therefore nowhere defined in the standard model, by Lemma 2.1 and the external equality of partial functions.

For a standard $n$, consider the theory

$$T_n := \mathsf{PA} + H_e(0, n).$$

If $T_n$ were inconsistent, the deduction theorem would yield $\mathsf{PA} \vdash \neg H_e(0, n)$, contrary to Lemma 2.2. Thus $T_n$ is consistent. By first-order completeness it has a model $M_n$. This model satisfies exactly the two requirements in (2). The construction treats $n = 0$ just as it treats every other natural number. □

### 2.3 What the proof does and does not require

The proof uses consistency, rather than $\Sigma_1$-soundness, to show that no refutation can exist. It uses only the forward implication "actual finite computation implies provable computation sentence." Consequently it remains valid for a consistent c.e. extension $T$ of PA even if $T$ is arithmetically unsound.

Only the *external* fixed-point equality $\varphi_e = Q(e, \cdot)$ is needed for this conclusion. If the search terminated, that equality would give an actual standard terminating computation of $e$, and finite-computation verification would provide its PA proof. There is no need, for the theorem as stated, to formalize within PA the whole proof of Kleene's recursion theorem.

Functionality of the fixed computation formula is also important for the meaning of the result:

$$\mathsf{PA} \vdash \forall a\, \forall x\, \forall y\, \forall z\, \big((\mathrm{Halt}(a, x, y) \wedge \mathrm{Halt}(a, x, z)) \to y = z\big).$$

It follows from determinism of the chosen operational semantics. The different prescribed standard outputs occur in different models.

**Corollary 2.4** (Independence of each output sentence)**.** *For every standard $n$, PA proves neither $H_e(0, n)$ nor its negation.*

*Proof.* Unprovability of the negation was established already. If PA proved $H_e(0, n)$, functionality and distinctness of the numerals for $n$ and $n + 1$ would give $\mathsf{PA} \vdash \neg H_e(0, n + 1)$, a contradiction. This argument still uses only consistency together with the usual functionality proof. □

## 3 Connection with the supplied universal-algorithm reference

Hamkins's exposition of Woodin's theorem supplies a program $u$ which can enumerate any prescribed finite sequence in an appropriate PA model. It also gives a stronger extension property for models; the single-output question needs only the finite-sequence assertion [1]. Define

$$f(x) := \text{simulate } u \text{ until its first enumerated number appears, then return that number.}$$

For each $n$, take a model in which $u$ enumerates exactly $[n]$. In that model $f(0) = n$. In the standard model $u$ enumerates nothing: otherwise a real finite enumeration event would be provable in PA, contradicting the existence of a PA model in which its sequence is empty. Thus $f(0)$ diverges standardly.

The direct proof in Section 2 is the one-input form of the self-reference method credited to Vadim Kosoy in Hamkins's earlier exposition [2]. It makes a smaller Lean target than formalizing the entire end-extension theorem. No claim of a new universal-algorithm theorem is being made here.

# 4 How the nonstandard model supports the computation

In a witness model $M_n$, the code $e$, the input 0, and the output $n$ are all standard numerals. The model's witness to the existential computation assertion is nonstandard. If there were a standard valid computation witness, its finite verification would show that the program really halts, contradicting the proof above.

For a literal step-by-step machine presentation, the first halting time must exceed every standard natural number of the model. Any prescribed standard number of deterministic steps starting from a standard configuration agrees with the real computation. The model regards its longer coded history as finite because its own natural-number domain contains nonstandard elements.

This explains why the theorem is compatible with determinism and the fixed value of every actually terminating standard computation. It also explains why Lean must keep two semantic levels separate:

- $y \in$ `Code.eval` $e\ x$ describes standard computation in the metatheory;
- $M \models H_e(x, y)$ interprets a first-order arithmetic sentence in a potentially nonstandard structure.

These are related on the standard natural-number model by a representation theorem. They must not be identified for an arbitrary model.

# 5 A concrete Lean formalization of the statement

## 5.1 Use first-order syntax and genuine model existence

The accompanying `lean/Statement.lean` stays within Foundation's arithmetic syntax, already used by ProveIt's `PAUndecidable` development [5]. This avoids translating between two different first-order formalizations.

The essential target is:

```
def MainClaim : Prop :=
  ∃ e : Nat.Partrec.Code,
    ¬ (Nat.Partrec.Code.eval e 0).Dom ∧
    ∀ n : ℕ,
      LO.FirstOrder.Satisfiable
        (insert (haltSentence e 0 n) PA)
```

Here `Satisfiable` is Foundation's existing semantic predicate for existence of a first-order model. It is not a placeholder predicate named "has a model." The program type itself certifies that an external partial recursive function is being discussed. The stronger `StrongClaim` in the source replaces divergence at 0 by divergence at every standard input.

## 5.2 One fixed universal computation formula

To make `haltSentence` concrete, the source uses a single universal partial evaluator on a two-element vector:

```
def universalEval : List.Vector ℕ 2 →. ℕ :=
  fun v => Nat.Partrec.Code.eval
    (Nat.Partrec.Code.ofNatCode (v.get 0)) (v.get 1)

noncomputable def universalGraph :
    ArithmeticSemisentence 3 :=
  codeOfPartrec' universalEval

noncomputable def haltSentence
    (c : Code) (x y : ℕ) : ArithmeticSentence :=
  universalGraph/[↑y, ↑(Encodable.encode c), ↑x]
```

The variable order is output, program code, input. Foundation's `codeOfPartrec'` chooses an arithmetic program representation; its standard correctness theorem is `codeOfPartrec'_spec` [6].

The choice is made once, yielding one finite formula for a chosen universal interpreter. The formula is then specialized by numeral substitution. A fixed finite syntactic object can be hardwired into a computable syntax transformer, even when the initial existence proof uses classical choice. Choosing an unrelated noncomputable formula separately for every program would not justify the computable proof-search construction.

This is a specified universal-interpreter presentation. Its standard equivalence to Mathlib's evaluator alone does not prove PA-equivalence to every other possible computation encoding. If a later implementation requires the exact bounded-`evaln` presentation, it should prove the additional internal equivalence or use that presentation directly.

### 5.3 Make the representation contract explicit

The source defines the following additional obligations as propositions:

- `UniversalEvalPartrec`: partial recursiveness of the universal evaluator.
- `UniversalGraphSigmaOne`: the chosen graph is $\Sigma_1^0$.
- `StandardCorrectness`: satisfaction of each closed graph instance by $\mathbb{N}$ is equivalent to membership in `Code.eval`.
- `FiniteProofAdequacy`: actual halting implies a PA proof of the corresponding sentence.
- `RefutationsRE`: the relation $(c, n) \mapsto \mathsf{PA} \vdash \neg\texttt{haltSentence}(c, 0, n)$ is recursively enumerable uniformly in both parameters.
- `FunctionalityInPAModels`: the chosen graph has at most one output for each program/input pair in every PA model, including nonstandard arguments.

The most informative concrete endpoint is therefore

```
def FullySpecifiedMainClaim : Prop :=
  StandardCorrectness ∧
  FunctionalityInPAModels ∧
  MainClaim
```

These definitions do not assert that their propositions have already been proved. They state exactly what the completed integration must establish.

# 6 Existing results located in ProveIt and its dependencies

The source audit used the following fixed snapshot, rather than an unversioned assumption about current APIs:

| Component | Revision or version |
|---|---|
| ProveIt | `08edc6964d8990d2230179a9d65355ac1eee7a9e` |
| Lean | `leanprover/lean4:v4.32.0` |
| Mathlib | `81a5d257c8e410db227a6665ed08f64fea08e997` |
| Foundation | `32e1a0956a8622fad067328ca1959729a7634428` |

Foundation is exposed by ProveIt's Lake configuration from its vendored submodule. The fetched sources, rather than only README descriptions, were inspected. The archive's `dependencies.json` records the links.

## 6.1 Computability and self-reference

`Mathlib.Computability.PartrecCode` provides:

```
Nat.Partrec.Code.fixed_point₂
  {f : Code → ℕ →. ℕ} (hf : Partrec₂ f) :
  ∃ c : Code, eval c = f c
```

It also provides `eval_part`, `evaln_sound`, `evaln_complete`, and `primrec_evaln`. In particular,

$$y \in \texttt{eval}\ c\ x \quad \Longleftrightarrow \quad \exists k\ y \in \texttt{evaln}\ k\ c\ x.$$

These results support both the direct partial-function proof and a bounded-trace arithmetization [4].

## 6.2 Arithmetic representation and numeral substitution

The inspected Foundation representation file contains active declarations `code`, `models_code`, `code_sigma_one`, `codeOfPartrec'_spec`, `codeOfREPred_spec`, and `re_complete`. The forward finite-verification step can use `LO.FirstOrder.Arithmetic.sigma_one_completeness` from `Foundation.FirstOrder.Arithmetic.R0.Basic`.

ProveIt's `PAUndecidable.lean` supplies a particularly close template for the effective syntax work. It proves `haltingSentenceCode_computable` and `haltingSentenceCode_eq_quote`, and includes `computable_of_sigmaOne_definable_functionâĆĄ`. The required new transformer adds code/output parameters and a negation to the same kind of numeral-substitution construction [5].

## 6.3 Consistency of an extension and existence of a model

Foundation's `Incompleteness.Dense` has `LO.Entailment.consistent_cons_of_unprovable_neg`. It turns unrefutability of a sentence into consistency after adjoining that sentence. Then `LO.FirstOrder.Theory.small_satisfiable_of_consistent`, in `Foundation.FirstOrder.Completeness.CounterModel`, supplies a genuine model [7]. The essential final step has the following shape:

```
-- For sigma = haltSentence e 0 n:
have hcon : Consistent (insert sigma PA) :=
  consistent_cons_of_unprovable_neg hnot
exact Theory.small_satisfiable_of_consistent hcon
```

Here `PA` is an explanatory ASCII alias for the source's PA notation; the exact namespaces and instances are recorded in the API audit. No new completeness theorem needs to be proved.

### 6.4 Two implementation hazards

**A commented theorem is not a reusable result.** In the inspected representation file, `codeAux_uniq` and `code_uniq` occur inside a block comment. They cannot be imported as proved declarations. Moreover, that commented sketch is stated over a weak theory while using global order comparability in its minimization case. The needed functionality lemma should be proved under PA (or adequate ordered-arithmetic axioms), rather than obtained by blindly uncommenting the old block.

**An unrestricted induction interface is too strong.** One other subsystem in ProveIt uses `SetTheory.PA.Model`, whose induction field ranges over every Lean predicate. That is not the first-order nonstandard-model interface needed here. The repository also has a genuine first-order bundle `LeanProofs.PAFiniteBasisReduction.TwoNonisomorphicModels.FirstOrderPAModel` [8]. For this project, staying with Foundation's `Satisfiable` avoids mixing the two syntaxes.

## 7 The supplied Lean proof files

### 7.1 The logic-only core

`lean/ProofCore.lean` imports only `Std`. It explicitly packages program evaluation, arithmetic sentences, provability, model satisfaction, consistency, finite computation verification, and completeness into an `ArithmeticInterface`. Its `SelfSearch` proposition has just the two operational properties used by the proof:

$$\varphi_e(0)\downarrow = y \Longrightarrow \mathsf{PA} \vdash \neg H_e(0, y),$$
$$(\exists n\ \mathsf{PA} \vdash \neg H_e(0, n)) \Longrightarrow \exists y\ \varphi_e(0)\downarrow = y.$$

It proves standard divergence, unrefutability, and existence of a model for every output. This small layer exposes the logical content without requiring the arithmetic coding library.

### 7.2 The assembly really uses the recursion theorem

`lean/MathlibAssembly.lean` is more concrete. Its program type is Mathlib's actual `Nat.Partrec.Code`, and standard evaluation is `Nat.Partrec.Code.eval`. The input data consist of an arithmetic bridge and an ordinary proof-search implementation. The search record contains:

```
search : Code → ℕ →. ℕ
partialRecursive : Partrec₂ search
ignoresInput : ∀ c x, search c x = search c 0
returnsRefutation : ∀ {c x y},
  y ∈ search c x → provable (neg (halts c 0 y))
findsRefutation : ∀ {c},
  (∃ n, provable (neg (halts c 0 n))) →
  ∃ m, m ∈ search c 0
```

The implementation of this search has no self-reference: $c$ is just a parameter. The assembly obtains self-reference by the actual library call

```
obtain ⟨e, he⟩ :=
  Nat.Partrec.Code.fixed_point₂ S.partialRecursive
```

and derives the claimed divergence and model existence from `he`. Thus the difficult fixed-point theorem is reused, rather than assumed as a new hypothesis about a desired witness.

The file concludes with a theorem whose shape is

```
∃ e : Code,
  Partrec (Nat.Partrec.Code.eval e) ∧
  (∀ x y, ¬ y ∈ Nat.Partrec.Code.eval e x) ∧
  (∀ n, ∃ M, A.isModel M ∧
    A.realizes M (A.halts e 0 n))
```

Another theorem names the partial function $f$ explicitly while retaining its code. All theorem bodies in these two files are supplied. Their arithmetic/search assumptions are explicit parameters. No claim is made that those parameters have already been instantiated for PA.

# 8 A clear route to a complete Lean proof

The remaining development can be divided into the following modules. The proposed module names are new project names, not claims about existing repository declarations.

## 8.1 Stage 1: validate the chosen computation presentation

In a proposed `PAFlexibleOutput.Computation` module:

1. Prove `UniversalEvalPartrec` from `eval_part`, computability of decoding, and vector projections. Convert to Foundation's vector-based `Nat.Partrec'` interface using `Nat.Partrec'.of_part`.
2. Apply `codeOfPartrec'_spec`. Simplify numeral substitution and the encoding round trip to prove `StandardCorrectness`.
3. Unfold the chosen representation and use `code_sigma_one` to prove that its graph and numeral instances are $\Sigma_1$.
4. Apply `sigma_one_completeness` to obtain `FiniteProofAdequacy`.

These steps use existing representation machinery. They should be verified independently of the self-reference argument.

## 8.2 Stage 2: prove functionality under PA

Prove that every fixed $\texttt{Nat.ArithPart}_1\texttt{.Code}$ compiled by Foundation's `codeAux`/`code` defines a partial function in every PA model. Structural induction on that finite syntax is enough. The arithmetic and projection cases are immediate. In a composition case, uniqueness of the intermediate tuple follows component by component from the induction hypotheses; uniqueness of the final result then follows from the outer induction hypothesis.

For minimization, suppose $y$ and $z$ are two candidate outputs. Each asserts that the inner program returns zero there and returns a nonzero value at every smaller argument. PA's linear ordering gives $y = z$, $y < z$, or $z < y$. In the $y < z$ case, the condition attached to $z$ provides a nonzero inner output at $y$, while the condition attached to $y$ provides zero. The inner uniqueness hypothesis contradicts this. The other strict inequality is symmetric.

Apply this lemma to the code selected for `universalGraph`. It proves the required internal functionality, recorded as `FunctionalityInPAModels`. This is a substantive missing formal lemma, but its mathematical proof is elementary and local. It should be established under PA's actual arithmetic/order instances.

### 8.3 Stage 3: implement uniform proof search

In a proposed `PAFlexibleOutput.ProofSearch` module:

1. Prove computability of the function sending $(c, n)$ to the Gödel code of $\neg H_c(0, n)$. Use the fixed finite formula and the numeral-substitution template from `PAUndecidable`.
2. Compose this transformer with the effective PA proof relation. This establishes the required r.e. relation. One may implement a direct decidable proof checker with explicit certificates, or use the recursively enumerable relation and dovetail its witnesses.
3. Implement the search over encoded pairs of a proof witness and an output numeral. Prove partial recursiveness by unbounded search over a computable acceptance test.
4. Prove `returnsRefutation`: every successful return retains the certificate it accepted.
5. Prove `findsRefutation`: a genuine finite proof has a standard finite code and is eventually reached. Its existence guarantees some return; it need not be the first accepted output.
6. Prove `ignoresInput`, since the ordinary input was not used by the search.

If starting from an r.e. relation rather than a decidable proof checker, the search must dovetail. Running a possibly nonterminating semidecision for each output sequentially would not be exhaustive.

### 8.4 Stage 4: instantiate logic and model existence

Use Foundation arithmetic sentences, PA provability, and its packaged first-order structures for the `ArithmeticBridge`. Obtain consistency from the standard natural-number model. Finite computation verification is Stage 1. For `modelOfUnrefuted`, combine the consistency-of-adjoining lemma with first-order completeness.

The result of `MathlibAssembly` now gives the concrete semantic conclusion. Convert "no output belongs to the partial value" into "its domain proposition is false" using the fact that every defined `Part` has its value as a member. Combining this with Stage 1 and Stage 2 proves `FullySpecifiedMainClaim`.

### 8.5 Stage 5: compile and audit

Check one module at a time in the pinned ProveIt environment. Its build instructions explicitly warn against concurrent Lean builds. The archive includes a sequential checking script for the supplied source files; it assumes their imported dependencies have already been built. Once the concrete PA instance is implemented, run `#print axioms` on the final theorem and verify that no admitted proof is present.

The remaining work is concentrated in arithmetic syntax and effective proof-search plumbing, plus the specified functionality lemma. It does not require a new recursion theorem, a new completeness theorem, or the full model-extension construction from Woodin's stronger result.

## 9 Optional strengthening: an internally total constant function

With a small extra implementation choice, the witness models can believe the program is total on their entire domains, including nonstandard inputs. Let $W(p)$ be an explicit wrapper which ignores its argument and runs $p$ at 0. Apply the recursion theorem to the operation which, on parameter $p$, searches for PA refutations of outputs of $W(p)$ at 0. If $p$ is the resulting fixed point and $e = W(p)$, the previous consistency argument still applies to $e$.

For a computation presentation in which the wrapper semantics have been verified in PA, we have

$$\mathsf{PA} \vdash \forall x \, \forall y \, (\mathrm{Halt}(\overline{e}, x, y) \leftrightarrow H_e(0, y)).$$

Consequently a model satisfying $H_e(0, n)$ satisfies $\forall x \; \mathrm{Halt}(\overline{e}, x, \overline{n})$: it regards $e$ as the total constant-$n$ function. Externally the program is still nowhere defined. This strengthening needs an internal wrapper-correctness lemma. The external fixed-point equality by itself does not automatically establish the internal equation, and the stronger theorem is not claimed as proved by the supplied Lean sources.

## 10 Delivery and verification status

| Item | Status |
|---|---|
| Mathematical theorem | Complete proof in Sections 1–2. |
| Connection to user reference | Explained using the first-output extractor. |
| Repository API audit | Relevant pinned source files fetched and inspected. |
| Concrete Lean statement | Defined in `Statement.lean`; representation requirements are explicit. |
| Logical core | Filled proof terms under explicit interfaces. |
| Recursion-theorem assembly | Filled proof terms applying Mathlib's `fixed_point`$_2$, conditional on arithmetic/search interfaces. |
| Concrete PA instance | Implementation tasks listed in Section 8; not yet completed. |
| Local Lean compilation | Not performed; no Lean or Lake executable was available. No successful kernel check is claimed. |
| PDF | Compiled from the included source. |

The archive contains the paper, the three Lean source files, exact dependency information, mathematical and API audit notes, and reproduction scripts. It is intended to be directly usable as the start of a focused `PAFlexibleOutput` development inside ProveIt.

## References


[1] Joel David Hamkins, *The universal algorithm: a new simple proof of Woodin's theorem*, May 28, 2017. User-supplied reference. https://jdh.hamkins.org/the-universal-algorithm-a-new-simple-proof-of-woodins-theorem/.

[2] Joel David Hamkins, *Every function can be computable!*, March 25, 2016, including the alternative proof credited to Vadim Kosoy. https://jdh.hamkins.org/every-function-can-be-computable/.

[3] Vladimir Reshetnikov, *ProveIt*, inspected at commit `08edc6964d8990d2230179a9d65355ac1eee7a9e`. https://github.com/VladimirReshetnikov/ProveIt/tree/08edc6964d8990d2230179a9d65355ac1eee7a9e.

[4] Mathlib, *Mathlib/Computability/PartrecCode.lean*, revision `81a5d257c8e410db227a6665ed08f64fea08e997`. https://github.com/leanprover-community/mathlib4/blob/81a5d257c8e410db227a6665ed08f64fea08e997/Mathlib/Computability/PartrecCode.lean. API documentation: https://leanprover-community.github.io/mathlib4_docs/Mathlib/Computability/PartrecCode.html.

[5] ProveIt, *Logic/PeanoArithmetic/Undecidable/Lean/PAUndecidable.lean*, at the revision in [3]. https://github.com/VladimirReshetnikov/ProveIt/blob/08edc6964d8990d2230179a9d65355ac1eee7a9e/Logic/PeanoArithmetic/Undecidable/Lean/PAUndecidable.lean.

[6] FormalizedFormalLogic/Foundation, *Foundation/FirstOrder/Arithmetic/R0/Representation.lean*, revision `32e1a0956a8622fad067328ca1959729a7634428`. https://github.com/FormalizedFormalLogic/Foundation/blob/32e1a0956a8622fad067328ca1959729a7634428/Foundation/FirstOrder/Arithmetic/R0/Representation.lean.

[7] FormalizedFormalLogic/Foundation, *Foundation/FirstOrder/Completeness/CounterModel.lean*, at the revision in [6]. https://github.com/FormalizedFormalLogic/Foundation/blob/32e1a0956a8622fad067328ca1959729a7634428/Foundation/FirstOrder/Completeness/CounterModel.lean.

[8] ProveIt, *PAFiniteBasisReduction/TwoNonisomorphicModels.lean*, at the revision in [3]; in `Logic/PeanoArithmetic/NotFinitelyAxiomatizable/Lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/08edc6964d8990d2230179a9d65355ac1eee7a9e/Logic/PeanoArithmetic/NotFinitelyAxiomatizable/Lean/PAFiniteBasisReduction/TwoNonisomorphicModels.lean.